

Market Mechanics, Data & Backtesting

Build the reusable infrastructure required to turn historical market data and strategy signals into realistic trades, P&L, and risk metrics.

You will be given **two datasets** — a daily OHLCV bar dataset and an order-book (LOB) snapshot dataset. Build **two separate backtesters**, one per dataset: a bar-based engine for the OHLCV data, and a book-based engine for the order-book data. Within each backtester, the requirements below (data pipeline, strategy interface, execution engine, P&L, risk analysis) apply independently.

Part 1 — Market Data & Backtesting Infrastructure

1. Market & Data Fundamentals

Before implementing the pipelines, understand:

- OHLCV data
- Tick data and order-book depth, and how they differ from OHLCV
- Timestamps and chronological ordering
- Simple returns and log returns
- Bid, ask and bid-ask spread
- Market orders vs limit orders
- Basic Limit Order Book (LOB)
- Slippage and transaction costs

2. Data Refinement Pipeline

Build a reusable data-refinement pipeline (one instance per dataset) that:

- Loads the raw data (OHLCV bars, or order-book snapshots).
- Converts timestamps into a standardized format (the OHLCV dataset ships with a few dates in a non-ISO format on purpose — normalize them).
- Sorts observations chronologically.
- Handles missing observations and duplicates appropriately.
- Allows selection of a particular timeframe and ensures all subsequent analysis uses only that timeframe.
- Calculates simple and log returns.
- Resamples data where required.
- Produces a clean DataFrame ready for backtesting.

The pipeline logic should be reusable rather than written specifically for one dataset — i.e. the same design pattern should apply whether you're pointed at the OHLCV file or the order-book file, even though the two backtesters that consume the output are separate.

3. Strategy Interface

Design the system so that the strategy and backtesting engine are independent. A strategy should take market data/features as input and generate signals such as:

```
+1 → Long  
0 → No position  
-1 → Short
```

The execution engine should independently interpret these signals and simulate the resulting trades.

4. Execution Engine & P&L

Build an execution pipeline (per backtester) that:

- Takes strategy signals as input.
- Converts signals into position changes.
- Simulates order execution.
- Accounts for transaction costs and slippage.
- Tracks cash and positions.
- Records every trade.
- Calculates realized and unrealized P&L.
- Generates the portfolio value / equity curve.

Transaction cost: apply 0.02% per side — 0.02% of trade notional on every buy, and 0.02% of trade notional on every sell (so a full round-trip costs 0.04% total). Apply this in both backtesters.

Slippage: for the OHLCV backtester, model slippage explicitly (e.g. a fixed bps assumption or execute at next-bar open rather than the signal bar's close). For the order-book backtester, slippage should emerge naturally from execution — fill against the actual bid/ask levels and their available volume (walk the book) rather than assuming a flat cost.

The engine should maintain enough trade-level information to answer: *What position was held, when was it changed, at what price was the trade executed, and how did it affect the portfolio?*

Important: Avoid look-ahead bias. A signal generated using information available at time t must not execute using information that was unavailable at t .

Part 2 — Performance & Risk Analysis

Calculate at least:

- Rolling volatility
- Rolling Sharpe ratio
- Rolling Sortino ratio
- Maximum drawdown
- Drawdown duration
- Calmar ratio

Also report basic statistics such as:

- Total return
- Annualized return
- Number of trades
- Win rate
- Average trade return
- Profit factor

Visualize the important results, including the equity curve and drawdown — for **each** of the two backtesters.

Part 3 — Baseline Strategy

Implement a simple strategy to validate each pipeline.

Example: Moving Average Crossover

- Short-term MA = 20 periods
- Long-term MA = 50 periods

Example rule:

```
Short MA > Long MA → Long
Short MA < Long MA → Short
```

Run this strategy through both complete pipelines (OHLCV close price for the bar backtester; mid-price, or another feature you derive from the book, for the order-book backtester).

The objective is not to optimize the strategy. The baseline is primarily a sanity check that the complete flow works correctly, for both backtesters:

```
Data → Strategy → Signals → Execution → Trades → P&L → Metrics
```

Expected Deliverables

- Two reusable data-cleaning and resampling pipelines (one per dataset).
- Two modular strategy interfaces.
- Two execution engines, each with the 0.02%-per-side transaction cost and an appropriate slippage model (fixed-assumption for OHLCV, book-walk-derived for the order book).
- Trade logs and portfolio/equity curves for both.
- P&L and risk-metric engines for both.
- Working baseline strategy on both datasets.
- Performance report and relevant visualizations for both.

Core requirement: Within each backtester, adding or replacing a strategy should not require rewriting that backtester's data-processing, execution, P&L, or performance-analysis components.